RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

Department of Computer Science and Engineering

CS23334 Fundamentals of Data Science Lab

III semester II Year (2023R)

Name of the Student: UMASANKAR.N

Register Number: 240701574

EXPERIMENT NO-1(A)

MATPLOTLIB LIBRARY-DATA VISUALLIZATION

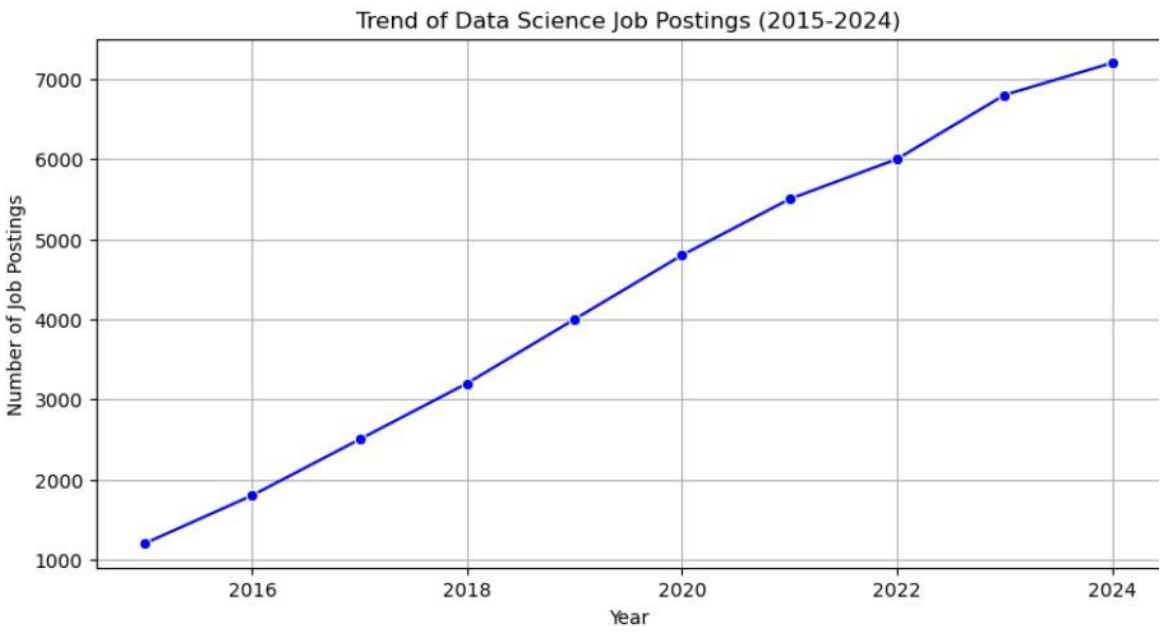
Aim: To analyze the trends of data science job posting over the last decade

Procedure:

1. Import NumPy and Matplotlib
2. Give some dummy data and make it as Data Frame
3. Use Bar plot for year over pasting
4. Give attributes like xlabels ,ylabel, title etc
5. Finally show line plot

Program:

[1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
data = {
    'Year': [2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024],
    'Job_Postings': [1200, 1800, 2500, 3200, 4000, 4800, 5500, 6000, 6800, 7200]
}
df = pd.DataFrame(data)
plt.figure(figsize=(10,5))
sns.lineplot(x='Year', y='Job_Postings', data=df, marker='o', color='blue')
plt.title("Trend of Data Science Job Postings (2015-2024)")
plt.xlabel("Year")
plt.ylabel("Number of Job Postings")
plt.grid(True)
plt.show()
```



Result:

Thus the python program to visualize data using line plot is executed successfully

EXPERIMENT NO-1(B)

MATPLOTLIB LIBRARY-DATA VISUALLIZATION

Aim:

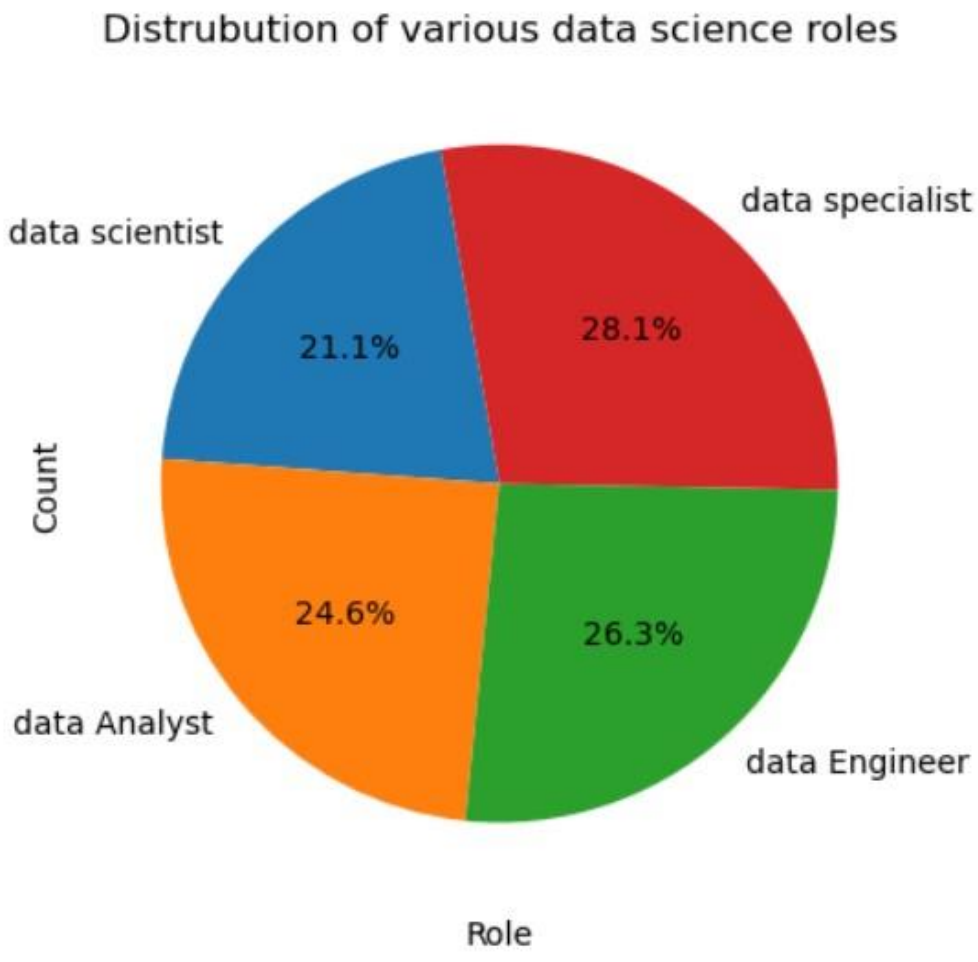
To analyze and visualize distribution of various of Data Science

Procedure:

1. Import pandas and Matplotlib
2. Create a dataset
3. Visualize through pie chart for job posting for various Data Science Roles
4. Use some attributes like colors,title,figure size etc

Program:

```
[6]: import pandas as pd
     import matplotlib.pyplot as plt
     import seaborn as sns
     roles_data={
         'Role':['data scientist','data Analyst','data Engineer','data specialist'],
         'count':[120,140,150,160]
     }
     df_roles=pd.DataFrame(roles_data)
     plt.pie(df_roles['count'],labels=df_roles['Role'],autopct='%1.1f%%',startangle=100)
     plt.title("Distrubution of various data science roles")
     plt.xlabel("Role")
     plt.ylabel("Count")
     plt.show()
```



Result:

Thus the python program to visualize data science roles using line plot is executed successfully

EXPERIMENT NO-1(C)

DISPLAY THE STRUCTURED, UNSTRUCTURED AND SEMI-STRUCTURED DATA

Aim:

To differentiate structured, unstructured and semi structured data.

Procedure:

1. Import Matplotlib and Pandas Create a Data Frame for structured data and print it.
2. Likewise create Data Frame for unstructured and normalize.
3. It using attribute and print it
4. Now create unstructured data and print it

Program:

[33]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
structured_data=pd.DataFrame(
    {
        'ID':[1,2,3],
        'Name':['Alice','Bob','Choice'],
        'Salary':[70000,80000,90000]
    }
)
unstructured_data={
    "This is an email text",
    "Image file:cat.png",
    "Audio recording:interview ,mp3"
}
semistructured_data=[
    {"Name":"Alice","Skills":["Python","ML"]},
    {"Name":"Bob","Skills":["SQL","Data Analyst"]}
]
print("Structured_data\n",structured_data);
print("Unstructured_data\n",unstructured_data);
print("semistructured_data\n",semistructured_data);
```

```
Structured_data
    ID    Name  Salary
0   1   Alice   70000
1   2     Bob   80000
2   3  Choice   90000
Unstructured_data
 {'Audio recording:interview ,mp3', 'This is an email text', 'Image file:cat.png'}
semistructured_data
 [{'Name': 'Alice', 'Skills': ['Python', 'ML']}, {'Name': 'Bob', 'Skills': ['SQL', 'Data Analyst']}]
```

Result:

Thus the python program of the structured, unstructured and semi-structured data executed successfully.

EXPERIMENT NO-1(D)

Study of Encrypt and decrypt sensitive data

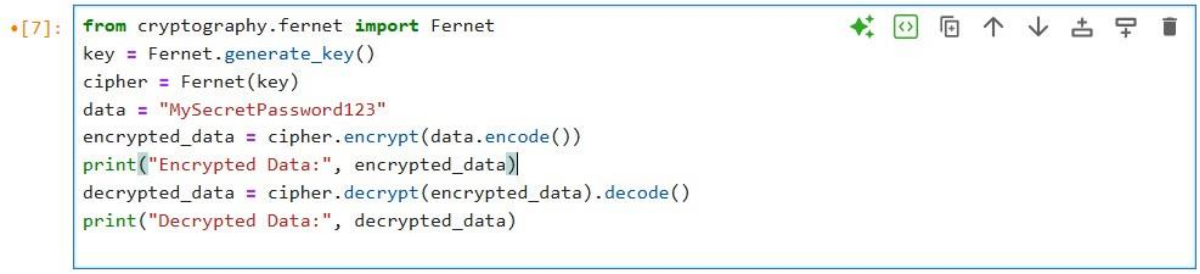
Aim:

To study the Encrypt and decrypt sensitive data.

Procedure: 1. Import pandas Data Frame with rows and columns.

2. using JSON, dictionaries, or lists without a fixed schema.
3. free text, images, or multimedia.
4. **formats, storage methods, and analysis ease.**
5. **Structured → easy tantalize, Semi-structured → flexible, Unstructured → needs preprocessing**.

Program:



```
[7]: from cryptography.fernet import Fernet
key = Fernet.generate_key()
cipher = Fernet(key)
data = "MySecretPassword123"
encrypted_data = cipher.encrypt(data.encode())
print("Encrypted Data:", encrypted_data)
decrypted_data = cipher.decrypt(encrypted_data).decode()
print("Decrypted Data:", decrypted_data)
```

```
Encrypted Data: b'gAAAAABo_bq-wjBU-gmlDrghHUbTZKhKD45a30v-9Q09AO5v7Xr-DdDsB-Kw7Zr4f4J8qr_OFGJQJKfnaQoP
cHyT6-c9t6Wr4OB1A_u3XERNjndzmJ2gRkA='
Decrypted Data: MySecretPassword123
```

Result:

Thus the python program of the Encrypt and Decrypt sensitive data executed successfully.

EXPERIMENT – 2

PANDAS LIBRARY – BASIC CONCEPT

Aim:

To upload and analyze dataset given in csv format and perform data preprocessing and visualization

Procedure:

1.Upload the csv file and read it

2. Import the necessities like pandas ,numpy and seaborn

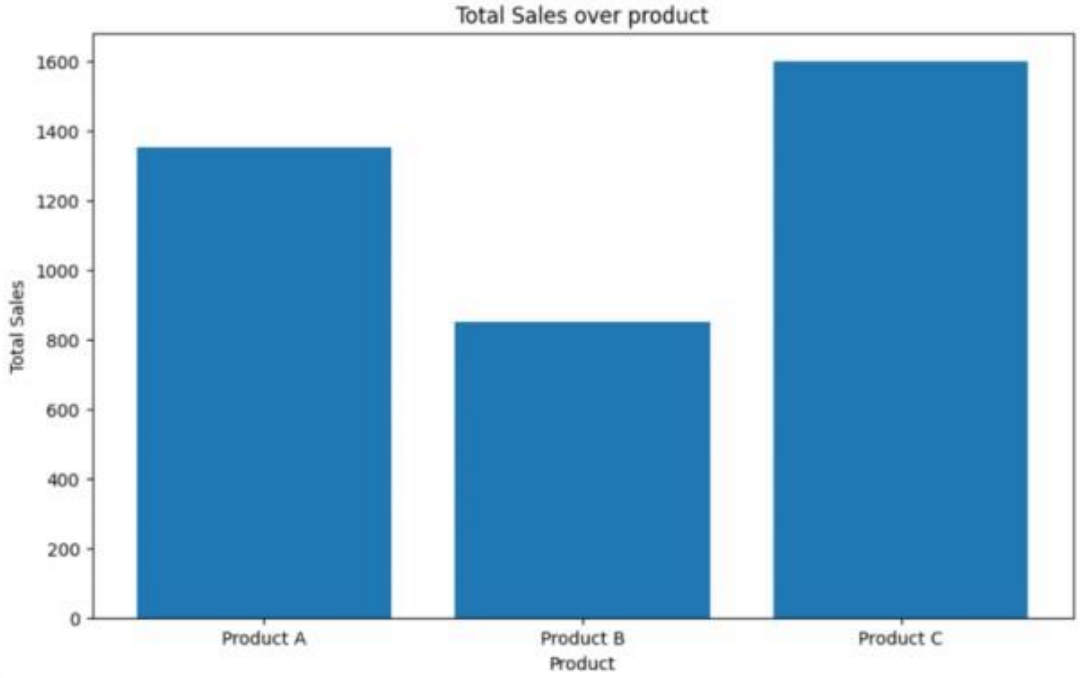
3.Now visualize bar plot for sales over products , line plot for sales over time and correlation matrix

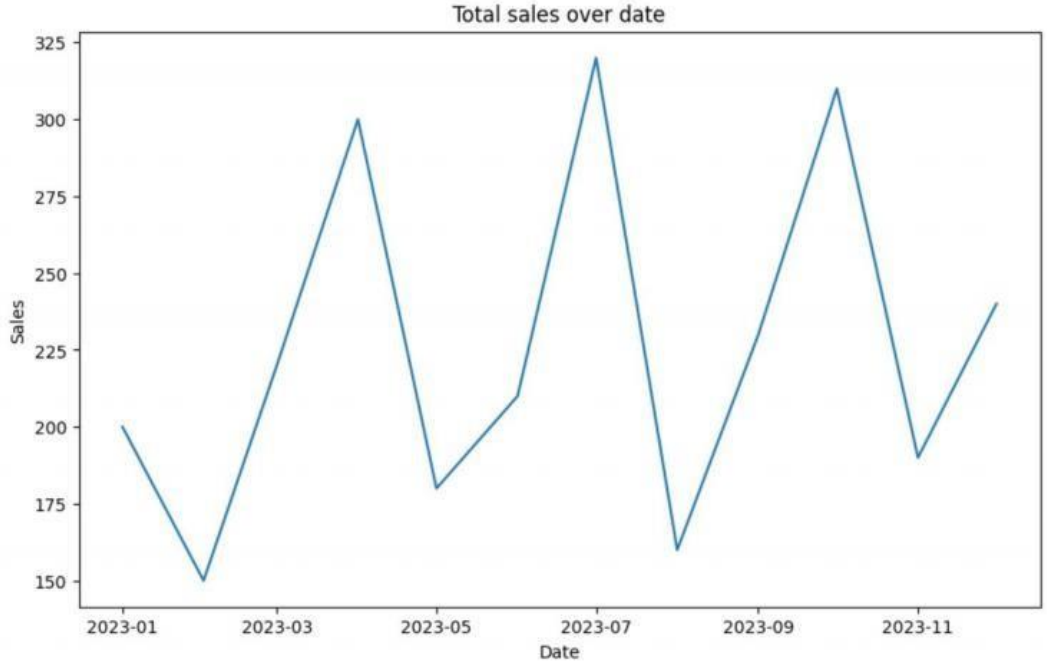
PROGRAM:

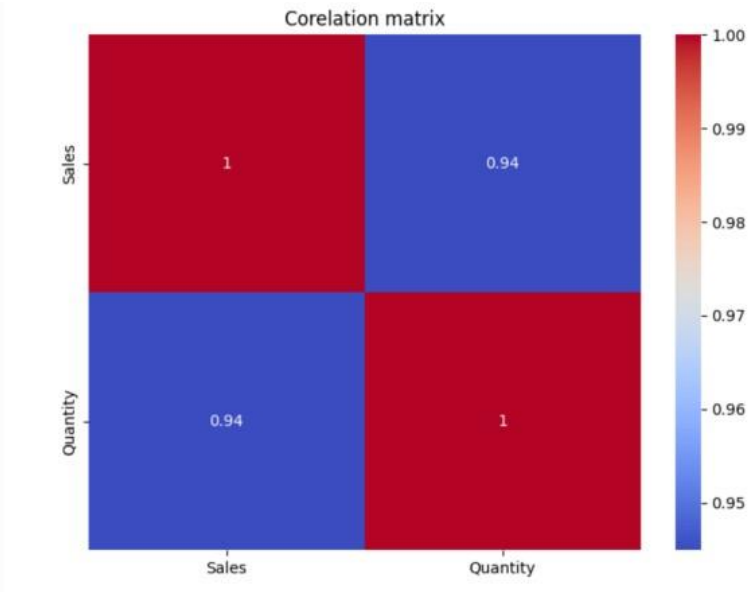
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Load the data into a pandas DataFrame
file_path='C:\sales_data.csv'
df = pd.read_csv(file_path)
# Display the first few rows of the DataFrame
print(df.head())
# Check for missing values
print(df.isnull().sum())
# Fill or drop missing values if necessary
df['Sales'].fillna(df['Sales'].mean(), inplace=True)
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)
# Summary statistics
print(df.describe())
# Group by product and calculate the total sales and quantity
product_summary = df.groupby('Product').agg({
'Sales': 'sum',
'Quantity': 'sum'
}).reset_index()
print(product_summary)

11

# Bar plot of total sales by product
plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
# Line plot of sales over time
df['Date'] = pd.to_datetime(df['Date'])
sales_over_time = df.groupby('Date').agg({'Sales': 'sum'}).reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'],sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('SalesOver Time')
```

```
plt.show()
# Line plot of sales over time
df['Date'] = pd.to_datetime(df['Date'])
sales_over_time = df.groupby('Date').agg({'Sales': 'sum'}).reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'],sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('SalesOver Time')
plt.show()
# Pivot table to analyze sales by region and product
pivot_table = df.pivot_table(values='Sales', index='Region', columns='Product',
aggfunc=np.sum, fill_value=0)
print(pivot_table)
# Correlation matrix
correlation_matrix = df.corr()
print(correlation_matrix)
# Heatmap of the correlation matrix
import seaborn as sns
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

```
Choose Files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving sales_data - Sheet1.csv to sales_data - Sheet1 (16).csv
         Date    Product  Sales  Quantity Region
0  01-01-2023  Product A    200         4  North
1  02-01-2023  Product B    150         3  South
2  03-01-2023  Product A    220         5  North
3  04-01-2023  Product C    300         6   East
4  05-01-2023  Product B    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
            Sales   Quantity
count   16.000000  16.000000
mean   237.500000   5.375000
std     64.031242   1.746425
min    150.000000   3.000000
25%    187.500000   4.000000
50%    225.000000   5.500000
75%    302.500000   7.000000
max    340.000000   8.000000
     Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
/tmp/ipython-input-1127875384.py:10: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an i
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behave

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) inst
```


Total Sales over product
Total Sales
1600
1400
1200
1000
800
600
400
200
0
Product A
Product B
Product C
Product


Total sales over date
Sales
325
300
275
250
225
200
175
150
2023-01
2023-03
2023-05
2023-07
2023-09
2023-11
Date



RESULT

Thus the python program to analyze the given dataset and perform data processing is executed and verified

EXPERIMENT – 3(A)

PANDAS LIBRARY – DATA PREPROCESSING

Aim:

To understand the importance of data preprocessing in data science

Procedure:

1.Upload the given dataset(csv file) and read it

2. Import necessities such as numpy ,pandas

3. Now , process the data , find missing values and replace it with mean(),mode(),median()

PROGRAM:

```
from google.colab import files
uploaded=files.upload()
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
file=next(iter(uploaded))
df=pd.read_csv(file)
df.Country.fillna(df.Country.mode()[0],inplace=True)
df.Age.fillna(df.Age.median(),inplace=True)
df.Salary.fillna(round(df.Salary.mean()),inplace=True)
pd.get_dummies(df.Country)
df.Purchased.replace(['Yes','No'],[1,0],inplace=True)
df
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	0
1	Spain	27.0	48000.0	1
2	Germany	30.0	54000.0	0
3	Spain	38.0	61000.0	0
4	Germany	40.0	63778.0	1
5	France	35.0	58000.0	1
6	Spain	38.0	52000.0	0
7	France	48.0	79000.0	1
8	Germany	50.0	83000.0	0
9	France	37.0	67000.0	1

Result:

Thus the python program to find missed values and replacing it was executed and verified

EXPERIMENT – 3(B)

PANDAS LIBRARY – HANDLING MISSING VALUES

Aim:

To handle and analyze missing and inappropriate data in dataset

Procedure:

1. Upload the csv file and read it
2. Import necessities such as pandas , numpy
3. Now check for missing and inappropriate data to replace with appropriate data

PROGRAM:

```
from google.colab import files
uploaded=files.upload()
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
file=next(iter(uploaded))
df=pd.read_csv(file)
df.drop_duplicates(inplace=True)
index=np.array(list(range(0,len(df))))
df.set_index(index,inplace=True)
df.drop(['Age_Group.1'],axis=1,inplace=True)
df.CustomerID.loc[df.CustomerID<0]=np.nan
df.Bill.loc[df.Bill<0]=np.nan
df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan
df.loc[(df['Rating(1-5)'] < 1) | (df['Rating(1-5)'] > 5), 'Rating(1-5)'] = np.nan
df['NoOfPax'].loc[(df['NoOfPax']<1)| (df['NoOfPax']>20)]=np.nan
df.FoodPreference.replace(['Vegetarian','veg'],'Veg',inplace=True)
df.FoodPreference.replace(['non-veg'],'Non-Veg',inplace=True)
df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()),inplace=True)
df.NoOfPax.fillna(round(df.NoOfPax.median()),inplace=True)
df.Bill.fillna(round(df.Bill.mean()),inplace=True)
df['Rating(1-5)'].fillna(df['Rating(1-5)'].median(),inplace=True)
df
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1.0	20-25	4.0	Ibis	Veg	1300.0	2.0	40000.0
1	2.0	30-35	5.0	LemonTree	Non-Veg	2000.0	3.0	59000.0
2	3.0	25-30	4.0	RedFox	Veg	1322.0	2.0	30000.0
3	4.0	20-25	4.0	LemonTree	Veg	1234.0	2.0	120000.0
4	5.0	35+	3.0	Ibis	Veg	989.0	2.0	45000.0
5	6.0	35+	3.0	Ibys	Non-Veg	1909.0	2.0	122220.0
6	7.0	35+	4.0	RedFox	Veg	1000.0	2.0	21122.0
7	8.0	20-25	4.0	LemonTree	Veg	2999.0	2.0	345673.0
8	9.0	25-30	2.0	Ibis	Non-Veg	3456.0	3.0	96755.0
9	10.0	30-35	5.0	RedFox	non-Veg	1801.0	4.0	87777.0

Result:

Thus we find missing and inappropriate data and replaced with appropriate ones

EXPERIMENT-3(C)

PANDAS LIBRARY – CREATE CSV FILE

Aim:

To create a dataset and make it as csv file and handle inappropriate values

Procedure:

1.Create a dataset

2. Import pandas and numpy and with its help, create data frame and change it to csv file

3. Read the dataset and handle inappropriate and missing values Program:

```
import pandas as pd
import numpy as np
data = {
    "Place_ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Name": [
        "Eiffel Tower", "Grand Canyon", "Great Barrier Reef", "Machu Picchu", "Mount Fuji",
        "Santorini", "Niagara Falls", "Petra", "Banff National Park", "Colosseum"
    ],
    "Country": [
        "France", "USA", "Australia", "Peru", "Japan",
        "Greece", "Canada", "Jordan", "Canada", "Italy"
    ],
    "Type": [
        "Monument", "Natural Wonder", "Natural Wonder", "Historical", "Mountain",
        "Island", "Waterfall", "Historical", "National Park", "Monument"
    ],
    "Average_Rating": [4.7, 4.8, 4.9, None, 4.6, 4.8, 4.7, 4.6, 4.9, None],
    "Entry_Fee": [25, 35, None, 45, 0, 10, None, 70, 20, 18],
    "Best_Season": [
        "Spring", "Fall", "Summer", "Spring", "Autumn",
        "Summer", "All year", "Winter", "Summer", "Spring"
    ]
}
df=pd.DataFrame(data)
df.to_csv('products.csv', index=False)
dff=pd.read_csv('products.csv')

dff
```

	Place_ID	Name	Country	Type	Average_Rating	Entry_Fee	Best_Season
0	1	Eiffel Tower	France	Monument	4.7	25.0	Spring
1	2	Grand Canyon	USA	Natural Wonder	4.8	35.0	Fall
2	3	Great Barrier Reef	Australia	Natural Wonder	4.9	NaN	Summer
3	4	Machu Picchu	Peru	Historical	NaN	45.0	Spring
4	5	Mount Fuji	Japan	Mountain	4.6	0.0	Autumn
5	6	Santorini	Greece	Island	4.8	10.0	Summer
6	7	Niagara Falls	Canada	Waterfall	4.7	NaN	All year
7	8	Petra	Jordan	Historical	4.6	70.0	Winter
8	9	Banff National Park	Canada	National Park	4.9	20.0	Summer
9	10	Colosseum	Italy	Monument	NaN	18.0	Spring

```
dff.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Place_ID        10 non-null     int64
 1   Name            10 non-null     object
 2   Country         10 non-null     object
 3   Type            10 non-null     object
 4   Average_Rating  8 non-null      float64
 5   Entry_Fee       8 non-null      float64
 6   Best_Season     10 non-null     object
dtypes: float64(2), int64(1), object(4)
memory usage: 692.0+ bytes
```

```
dff.Average_Rating.fillna(df.Average_Rating.median(),inplace=True)
dff.Entry_Fee.fillna(df.Entry_Fee.median(),inplace=True)
dff
```

	Place_ID	Name	Country	Type	Average_Rating	Entry_Fee	Best_Season
0	1	Eiffel Tower	France	Monument	4.70	25.0	Spring
1	2	Grand Canyon	USA	Natural Wonder	4.80	35.0	Fall
2	3	Great Barrier Reef	Australia	Natural Wonder	4.90	22.5	Summer
3	4	Machu Picchu	Peru	Historical	4.75	45.0	Spring
4	5	Mount Fuji	Japan	Mountain	4.60	0.0	Autumn
5	6	Santorini	Greece	Island	4.80	10.0	Summer
6	7	Niagara Falls	Canada	Waterfall	4.70	22.5	All year
7	8	Petra	Jordan	Historical	4.60	70.0	Winter
8	9	Banff National Park	Canada	National Park	4.90	20.0	Summer
9	10	Colosseum	Italy	Monument	4.75	18.0	Spring

```
dff.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Place_ID        10 non-null     int64
 1   Name            10 non-null     object
 2   Country         10 non-null     object
 3   Type            10 non-null     object
 4   Average_Rating  10 non-null     float64
 5   Entry_Fee       10 non-null     float64
 6   Best_Season     10 non-null     object
dtypes: float64(2), int64(1), object(4)
memory usage: 692.0+ bytes
```

Result:

Thus the python program to create own dataset and change it to csv file is executed and verified

EXPERIMENT – 4

DETECT OUTLIERS IN A GIVEN DATASET

Aim:

To detect outliers in a given dataset

Procedure:

1.Import numpy and create an array with random integers

2.Create a function for outlier

3.Then plot it using seaborn with displot and distplot

Program:

```
import numpy as np
array=np.random.randint(1,100,16)
array
```

```
array([18, 38, 76, 45, 93, 92, 73, 13, 83, 97, 15,  1,  4, 62, 29, 41])
```

```
array.mean()
```

```
np.float64(48.75)
```

```
np.percentile(array,25)
```

```
np.float64(17.25)
```

```
np.percentile(array,50)
```

```
np.float64(43.0)
```

```
np.percentile(array,75)
```

```
np.float64(77.75)
```

```
np.percentile(array,100)
```

```
np.float64(97.0)
```

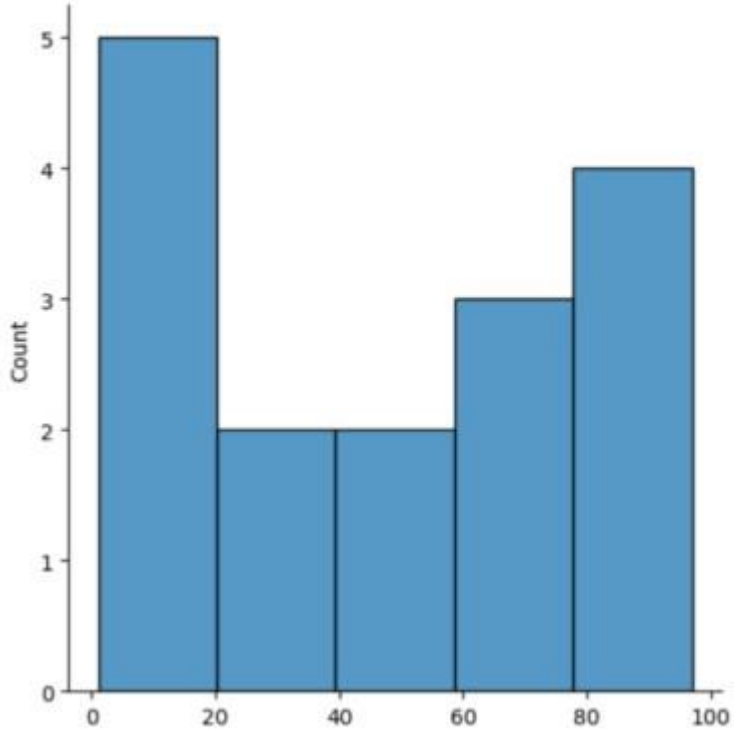
```
def outlierdetect(array):
  sorted(array)
  q1,q3=np.percentile(array,[25,75])
  iqr=q3-q1
  lr=q1-(1.5*iqr)
  ur=q3+(1.5*iqr)
  return lr,ur
```

```
lr,ur=outlierdetect(array)
lr,ur
```

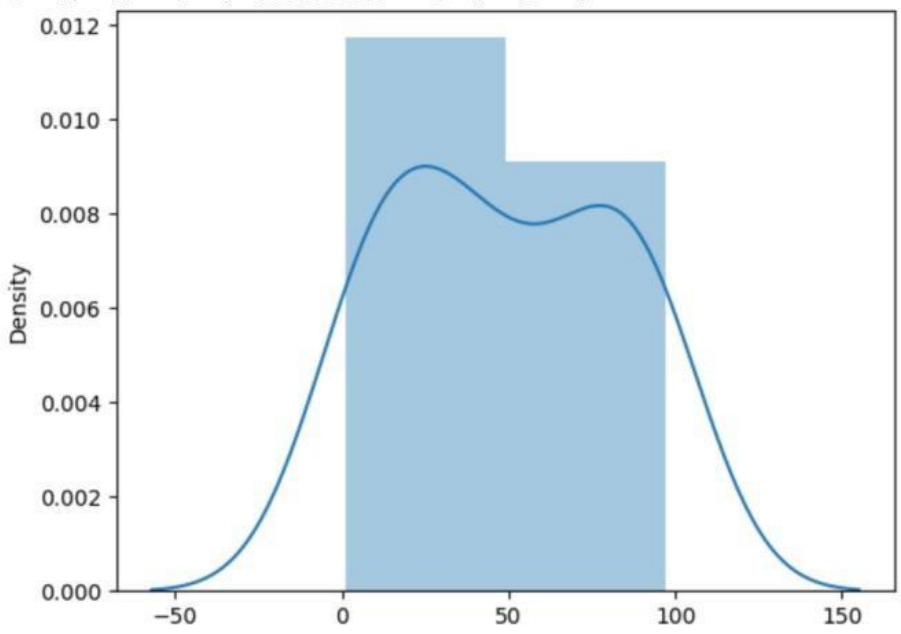
```
(np.float64(-73.5), np.float64(168.5))
```

```
import seaborn as sns
%matplotlib inline
sns.displot(array)
```

```
<seaborn.axisgrid.FacetGrid at 0x7bffd30a3e60>
```




```
sns.distplot(array)
```



Result:

Thus the python program to detect outliers in a given dataset is executed and output verified successfully

EXPERIMENT – 5

FEATURE SCALING IN GIVEN DATASET

Aim:

To provide feature scaling for a given dataset

Procedure:

1.Upload the given csv file

2. Import the necessities like numpy and Pandas

3. Use pandas to read_csv and make it as an data frame

4.Now use SimpleImputer to fill missing values

Program:

```
from google.colab import files
uploaded=files.upload()
import numpy as np
import pandas as pd
file=next(iter(uploaded))
df=pd.read_csv(file)
df
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving pre_process_datasample - pre_process_datasample (1).csv to pre_process_datasample - pre_process_datasample (1) (1).csv
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

1 / 8

```
df.head()
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes

```
df.Country.fillna(df.Country.mode()[0],inplace=True)
features=df.iloc[:,:-1].values
df.Country.fillna(df.Country.mode()[0],inplace=True)
label=df.iloc[:,-1].values
from sklearn.impute import SimpleImputer
age=SimpleImputer(strategy="mean",missing_values=np.nan)
Salary=SimpleImputer(strategy="mean",missing_values=np.nan)
age.fit(features[:,[1]])
SimpleImputer()
Salary.fit(features[:,[2]])
SimpleImputer()
```

SimpleImputer

```
SimpleImputer()
```

```
features[:,[1]]=age.transform(features[:,[1]])
features[:,[2]]=Salary.transform(features[:,[2]])
features
```

```
array([['France', 44.0, 72000.0],
       ['Spain', 27.0, 48000.0],
       ['Germany', 30.0, 54000.0],
       ['Spain', 38.0, 61000.0],
       ['Germany', 40.0, 63777.77777777778],
       ['France', 35.0, 58000.0],
       ['Spain', 38.77777777777778, 52000.0],
       ['France', 48.0, 79000.0],
       ['Germany', 50.0, 83000.0],
       ['France', 37.0, 67000.0]], dtype=object)
```

```
from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(sparse_output=False)
Country=oh.fit_transform(features[:,[0]])
Country
```

```
array([[1., 0., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [1., 0., 0.]])
```

```
final_set=np.concatenate((Country,features[:,[1,2]]),axis=1)
final_set
```

```
array([[1.0, 0.0, 0.0, 44.0, 72000.0],
       [0.0, 0.0, 1.0, 27.0, 48000.0],
       [0.0, 1.0, 0.0, 30.0, 54000.0],
       [0.0, 0.0, 1.0, 38.0, 61000.0],
       [0.0, 1.0, 0.0, 40.0, 63777.77777777778],
       [1.0, 0.0, 0.0, 35.0, 58000.0],
       [0.0, 0.0, 1.0, 38.77777777777778, 52000.0],
       [1.0, 0.0, 0.0, 48.0, 79000.0],
       [0.0, 1.0, 0.0, 50.0, 83000.0],
       [1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
sc.fit(final_set)
feat_standard_scaler=sc.transform(final_set)
feat_standard_scaler
```

```
array([[ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         7.58874362e-01,  7.49473254e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -1.71150388e+00, -1.43817841e+00],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
        -1.27555478e+00, -8.91265492e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -1.13023841e-01, -2.53200424e-01],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
         1.77608893e-01,  6.63219199e-16],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
        -5.48972942e-01, -5.26656882e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
         0.00000000e+00, -1.07356980e+00],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         1.34013983e+00,  1.38753832e+00],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
         1.63077256e+00,  1.75214693e+00],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
```

```
from sklearn.preprocessing import MinMaxScaler
mms=MinMaxScaler(feature_range=(0,1))
mms.fit(final_set)
feat_minmax_scaler=mms.transform(final_set)
feat_minmax_scaler
```

```
array([[1.        , 0.        , 0.        , 0.73913043, 0.68571429],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.13043478, 0.17142857],
       [0.        , 0.        , 1.        , 0.47826087, 0.37142857],
       [0.        , 1.        , 0.        , 0.56521739, 0.45079365],
       [1.        , 0.        , 0.        , 0.34782609, 0.28571429],
       [0.        , 0.        , 1.        , 0.51207729, 0.11428571],
       [1.        , 0.        , 0.        , 0.91304348, 0.88571429],
       [0.        , 1.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.43478261, 0.54285714]])
```

Result:

Thus the python program to do feature scaling in given dataset is executed and verified successfully

EXPERIMENT – 6

TO UNDERSTAND EDA – QUANTITATIVE AND QUALITATIVE

Aim:

To understand quantitative and qualitative of EDA

Procedure:

1. Import all the necessities
2. Upload default dataset ‘tips’
3. Use pandas to read and make it as Data Frame
4. Then perform various plots and joints using seaborn and other libraries

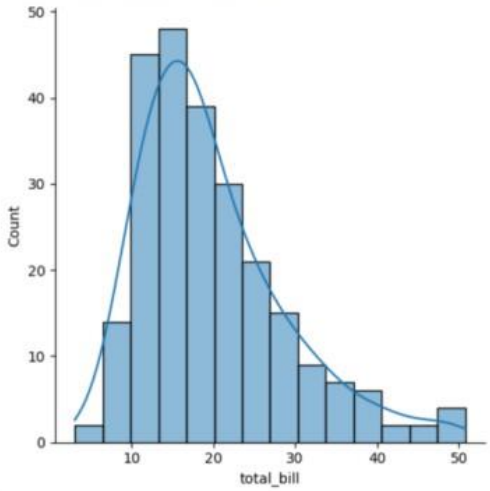
Program

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
tips=sns.load_dataset('tips')
tips.head()
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

```
sns.displot(tips.total_bill,kde=True)
```

```
<seaborn.axisgrid.FacetGrid at 0x7e56bd573ec0>
```



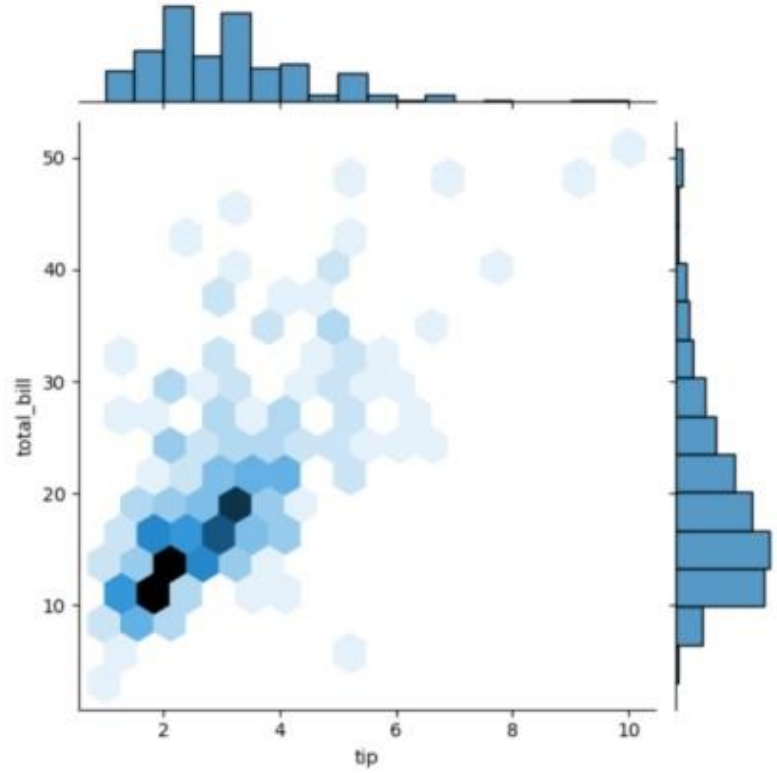

```
sns.displot(tips.total_bill,kde=False)
```

```
sns.jointplot(x=tips.tip,y=tips.total_bill)
```

```
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="reg")
```

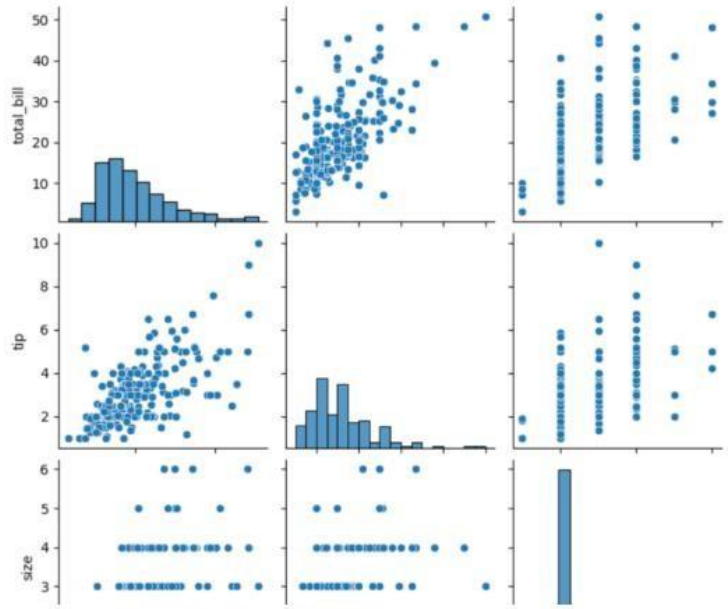
```
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="hex")
```

```
<seaborn.axisgrid.JointGrid at 0x7e569565aea0>
```



```
sns.pairplot(tips)
```

```
<seaborn.axisgrid.PairGrid at 0x7e5695577740>
```



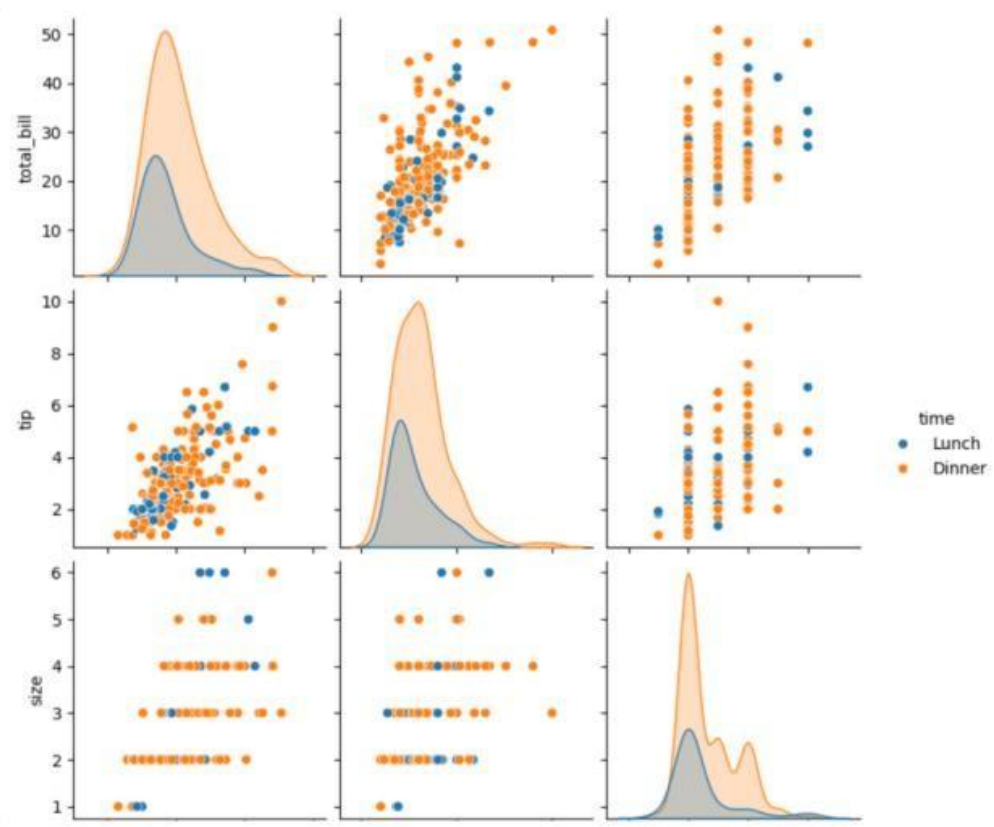

```
tips.time.value_counts()
```

time	count
Dinner	176
Lunch	68

dtype: int64

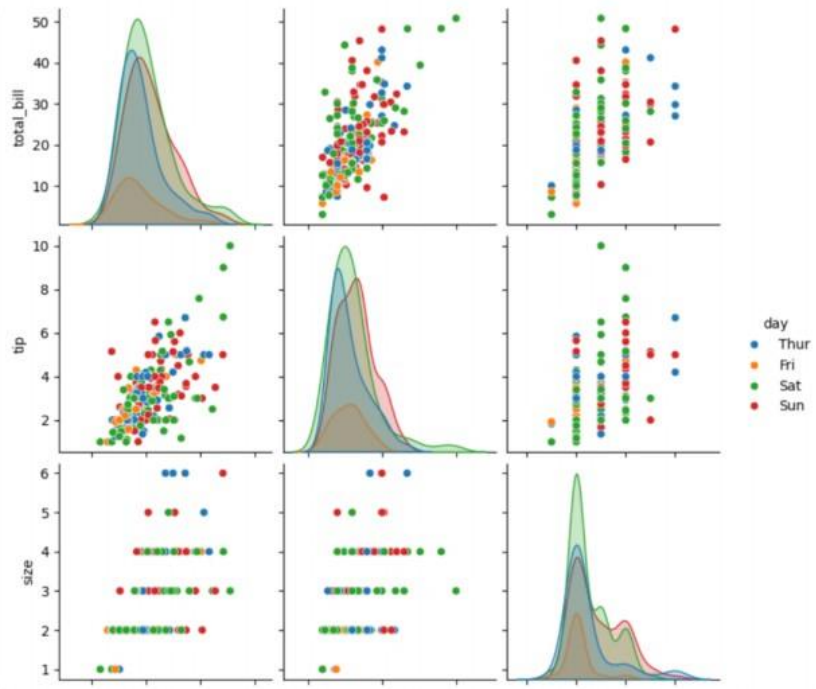
```
sns.pairplot(tips,hue='time')
```

```
<seaborn.axisgrid.PairGrid at 0x7e56941ba1e0>
```



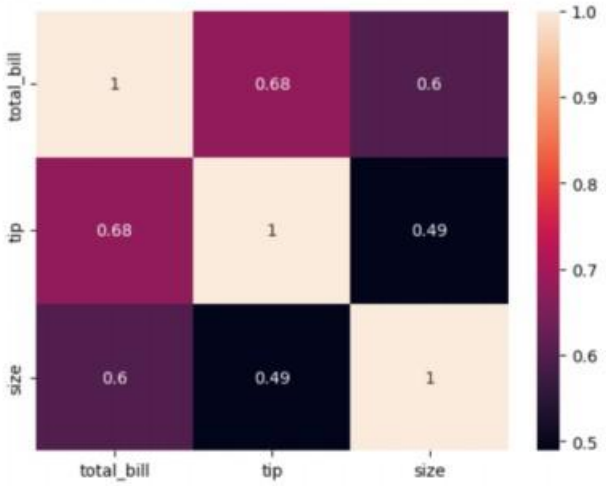

```
sns.pairplot(tips,hue='day')
```

```
<seaborn.axisgrid.PairGrid at 0x7e56955ca1e0>
```



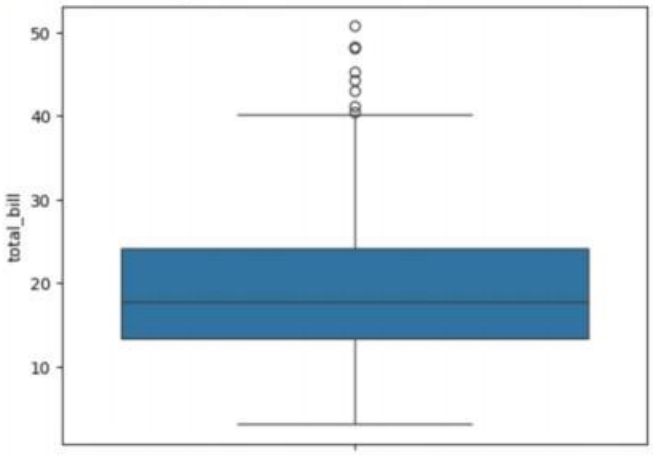
```
sns.heatmap(tips.corr(numeric_only=True),annot=True)
```

```
<Axes: >
```



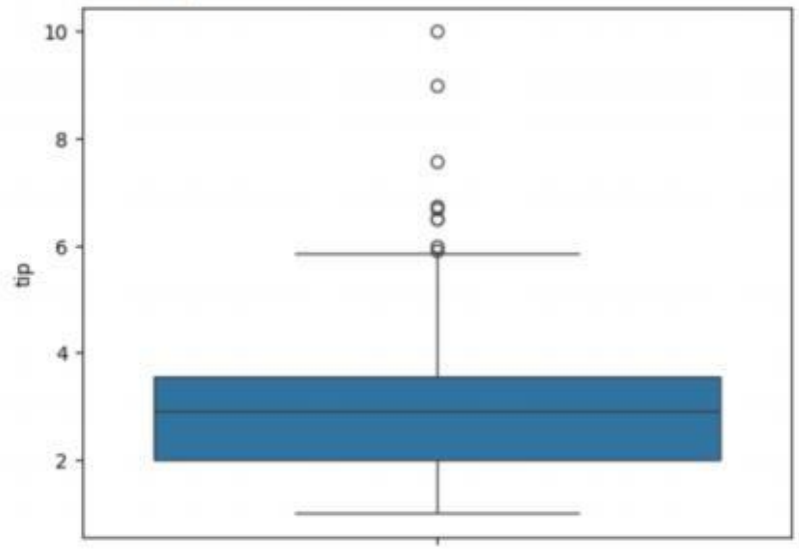

```
sns.boxplot(tips.total_bill)
```

```
<Axes: ylabel='total_bill'>
```



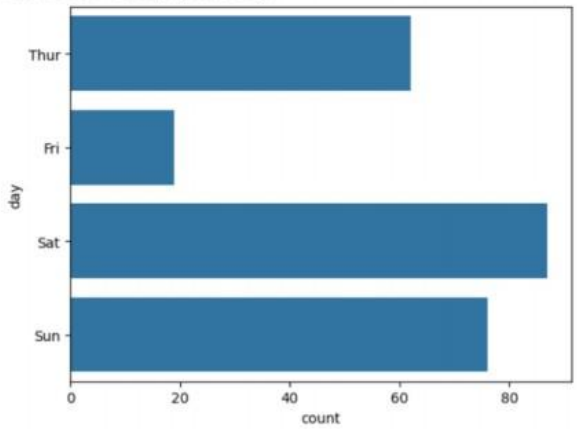
```
[ ]
✓ 0s
sns.boxplot(tips.tip)
```

```
<Axes: ylabel='tip'>
```



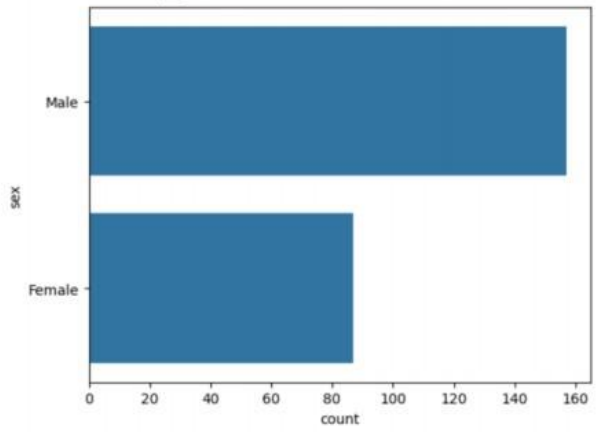
```
[ ]
✓ 0s
sns.countplot(tips.day)
```

```
<Axes: xlabel='count', ylabel='day'>
```



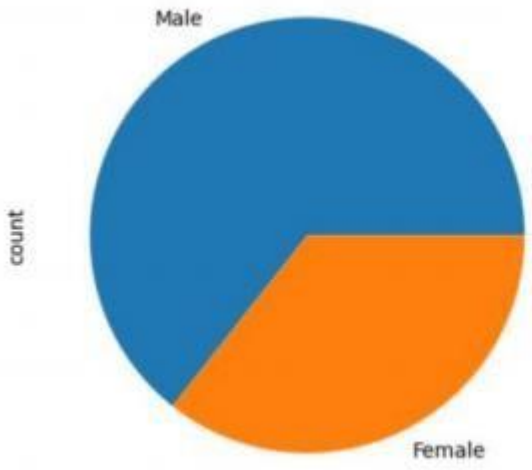

```
[ ]
✓ 0s
sns.countplot(tips.sex)
```

```
<Axes: xlabel='count', ylabel='sex'>
```

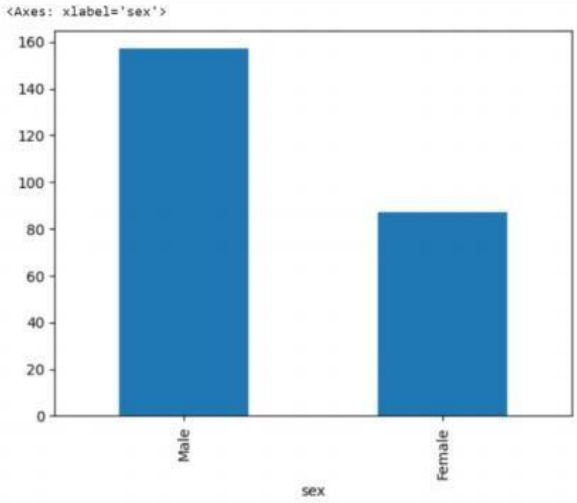



```
[ ]
✓ 0s
tips.sex.value_counts().plot(kind='pie')
```

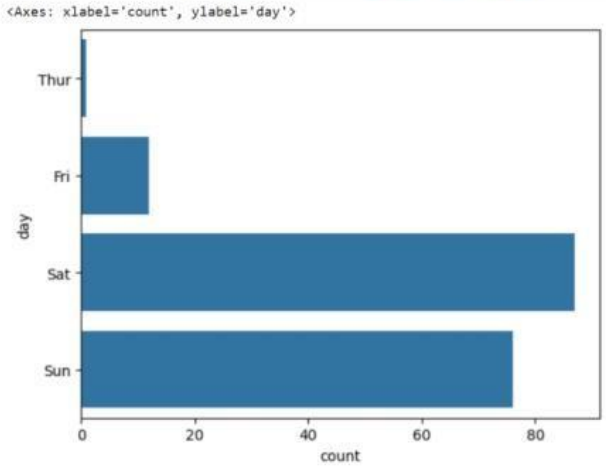
```
<Axes: ylabel='count'>
```



```
tips.sex.value_counts().plot(kind='bar')
```




```
sns.countplot(tips[tips.time=='Dinner']['day'])
```



Result:

Thus the python program to understand EDA is executed and verified successfully

EXPERIMENT – 7

PREDICTING MODEL - LINEAR REGRESSION

Aim:

To perform salary prediction model using Linear Regression

Procedure:

1. Upload the given dataset.
2. Import all the necessities

3. Read through the dataset and make it as dataframe
4. Through sklearn train the model
5. Test the mode

Program:

```
[ ]
✓ 9s
```

```
from google.colab import files
uploaded=files.upload()
import numpy as np
import pandas as pd
file=next(iter(uploaded))
df=pd.read_csv(file)
df
```

	YearsExperience	Salary
0	1.1	39343
1	1.3	46205
2	1.5	37731
3	2.0	43525
4	2.2	39891
5	2.9	56642
6	3.0	60150
7	3.2	54445
8	3.2	64445
9	3.7	57189
10	3.9	63218
11	4.0	55794
12	4.0	56957
13	4.1	57081
14	4.5	61111
15	4.9	67938

16	5.1	66029
17	5.3	83088
18	5.9	81363
19	6.0	93940
20	6.8	91738
21	7.1	98273
22	7.9	101302
23	8.2	113812
24	8.7	109431
25	9.0	105582
26	9.5	116969
27	9.6	112635
28	10.3	122391
29	10.5	121872

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

```
df.dropna(inplace=True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

```
df.describe()
```

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
max	10.500000	122391.000000

```
features=df.iloc[:,[0]].values
labels=df.iloc[:,[1]].values
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,labels,test_size=0.2,random_state=42)
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
```

```
LinearRegression
LinearRegression()
```

```
model.score(x_train,y_train)
```
```
0.9645401573418146
```

```
model.score(x_test,y_test)
```
```
0.9024461774180497
```

```
model.coef_
```
```
array([[9423.81532303]])
```

```
model.intercept_
```
```
array([25321.58301178])
```

```
filename = list(uploaded.keys())[0]
import pickle
pickle.dump(model,open(filename,'wb'))
model=pickle.load(open(filename,'rb'))
yr_of_exp=float(input("Enter years of experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)
```
```
Enter years of experience: 34
```

```
print("Estimated Salary for {} years of experience is {}: " .format(yr_of_exp,Salary))
```
```
Estimated Salary for 34.0 years of experience is [[345731.30399483]]:
```

Result:

Thus the python program for predicting model using Linear Regression is executed and verified

EXPERIMENT – 8

PREDICTING MODEL -KNN Aim:

To perform model classification using K-nearest neighbours

Procedure:

1. Upload a given dataset
2. Import all necessities
3. Read and make it as data frame
4. Through sklearn train the model
5. Test the model

Program:

```
from google.colab import files
uploaded=files.upload()
import numpy as np
import pandas as pd
file=next(iter(uploaded))
df=pd.read_csv(file)
df
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Iris (1) - Iris (1).csv to Iris (1) - Iris (1).csv
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Virginica
146	6.3	2.5	5.0	1.9	Virginica
147	6.5	3.0	5.2	2.0	Virginica
148	6.2	3.4	5.4	2.3	Virginica

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object 
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
df.variety.value_counts()
```

variety	count
Setosa	50
Versicolor	50
Virginica	50

dtype: int64

```
df.head()
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

```
features=df.iloc[:,:-1].values
labels=df.iloc[:,4].values
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
xtrain,xtest,ytrain,ytest=train_test_split(features,labels,test_size=.2,random_state=42)
model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)
```

```
KNeighborsClassifier()
```

```
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
```

```
0.9666666666666667
1.0
```

```
from sklearn.metrics import confusion_matrix
confusion_matrix(labels,model_KNN.predict(features))
```

```
array([[50,  0,  0],
       [ 0, 47,  3],
       [ 0,  1, 49]])
```

```
from sklearn.metrics import classification_report
print(classification_report(labels,model_KNN.predict(features)))
```

```
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.98      0.94      0.96        50
   Virginica       0.94      0.98      0.96        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

Result:

Thus the python program to perform model classification using KNN is executed and verified

EXPERIMENT – 9

LOGISTIC REGRESSION Aim:

To perform model classification using Logistic Regression

Procedure:

1. Upload the given dataset

2. Import all the necessities
3. Read and make it as Data Frame.
4. Through sklearn and train the model
5. Test the model

Program:

```
from google.colab import files
uploaded=files.upload()
import numpy as np
import pandas as pd
file=next(iter(uploaded))
df=pd.read_csv(file)
df
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Social_Network_Ads - Social_Network_Ads.csv to Social_Network_Ads - Social_Network_Ads.csv
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

```
df.head()
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
features=df.iloc[:,[2,3]].values
labels=df.iloc[:,4].values
features
```

```
array([[    19,  19000],
       [    35,  20000],
       [    26,  43000],
       [    27,  57000],
       [    19,  76000],
       [    27,  58000],
       [    27,  84000],
       [    32, 150000],
       [    25,  33000],
       [    35,  65000],
       [    26,  80000],
       [    26,  52000],
       [    20,  86000],
       [    32,  18000],
       [    18,  82000],
       [    29,  80000],
       [    47,  25000],
       [    45,  26000],
       [    46,  28000],
       [    48,  29000],
       [    45,  22000],
       [    47,  49000],
       [    48,  41000],
       [    45,  22000],
       [    46,  23000],
       [    47,  20000],
       [    49,  28000],
       [    47,  30000],
       [    29,  43000],
       [    31,  18000],
       [    31,  74000],
       [    27, 137000],
       [    21,  16000],
       [    28,  44000],
       [    27,  90000],
       [    35,  27000],
       [    33,  28000],
       [    30,  49000],
       [    26,  72000],
       [    27,  31000],
       [    27,  17000],
       [    33,  51000],
       [    35, 108000],
       [    30,  15000],
       [    28,  84000],
       [    23,  20000],
       [    25,  79000],
       [    27,  54000],
       [    30, 135000],
       [    31,  89000],
       [    24,  32000],
       [    18,  44000],
       [    29,  83000],
       [    35,  23000],
       [    27,  58000],
       [    24,  55000],
       [    35,  48000],
       [    22,  18000],
       [    32, 117000],
       [    27,  20000],
       [    25,  87000],
       [    23,  66000],
       [    32, 120000],
       [    59,  83000],
       [    24,  58000],
       [    24,  19000],
       [    23,  82000],
       [    22,  63000],
       [    31,  68000],
       [    25,  80000],
       [    24,  27000],
       [    20,  23000],
       [    33, 113000],
       [    32,  18000],
       [    34, 112000],
       [    18,  52000],
       [    22,  27000],
       [    28,  87000],
       [    26,  17000],
       [    30,  80000],
       [    39,  42000],
       [    20,  49000],
       [    35,  88000],
       [    30,  62000],
       [    31, 118000],
       [    24,  55000],
       [    28,  85000],
       [    26,  81000],
```

```
[    60,  46000],
[    60,  83000],
[    39,  73000],
[    59, 130000],
[    37,  80000],
[    46,  32000],
[    46,  74000],
[    42,  53000],
[    41,  87000],
[    58,  23000],
[    42,  64000],
[    48,  33000],
[    44, 139000],
[    49,  28000],
[    57,  33000],
[    56,  60000],
[    49,  39000],
[    39,  71000],
[    47,  34000],
[    48,  35000],
[    48,  33000],
[    47,  23000],
[    45,  45000],
[    60,  42000],
[    39,  59000],
[    46,  41000],
[    51,  23000],
[    50,  20000],
[    36,  33000],
[    49,  36000]])
```

[ ] ✓ 0s

```
labels
```

```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0,
       1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0,
       1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0,
       1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1,
       1, 1, 0, 1])
```

[ ] ✓ 9s

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
for i in range(1,401):
  x_train,x_test,y_train,y_test=train_test_split(features,labels,test_size=0.2,random_state=42)
  model=LogisticRegression()
  model.fit(x_train,y_train)
  train_score=model.score(x_train,y_train)
  test_score=model.score(x_test,y_test)
  if test_score>train_score:
    print("Test {} Train {} RandomState {}".format(test_score,train_score,i))
```

```
Test 0.8875 Train 0.8375 RandomState 1
Test 0.8875 Train 0.8375 RandomState 2
Test 0.8875 Train 0.8375 RandomState 3
Test 0.8875 Train 0.8375 RandomState 4
Test 0.8875 Train 0.8375 RandomState 5
Test 0.8875 Train 0.8375 RandomState 6
Test 0.8875 Train 0.8375 RandomState 7
Test 0.8875 Train 0.8375 RandomState 8
Test 0.8875 Train 0.8375 RandomState 9
Test 0.8875 Train 0.8375 RandomState 10
Test 0.8875 Train 0.8375 RandomState 11
Test 0.8875 Train 0.8375 RandomState 12
Test 0.8875 Train 0.8375 RandomState 13
Test 0.8875 Train 0.8375 RandomState 14
Test 0.8875 Train 0.8375 RandomState 15
Test 0.8875 Train 0.8375 RandomState 16
Test 0.8875 Train 0.8375 RandomState 17
Test 0.8875 Train 0.8375 RandomState 18
Test 0.8875 Train 0.8375 RandomState 19
Test 0.8875 Train 0.8375 RandomState 20
```

```
Test 0.8875 Train 0.8375 RandomState 371
Test 0.8875 Train 0.8375 RandomState 372
Test 0.8875 Train 0.8375 RandomState 373
Test 0.8875 Train 0.8375 RandomState 374
Test 0.8875 Train 0.8375 RandomState 375
Test 0.8875 Train 0.8375 RandomState 376
Test 0.8875 Train 0.8375 RandomState 377
Test 0.8875 Train 0.8375 RandomState 378
Test 0.8875 Train 0.8375 RandomState 379
Test 0.8875 Train 0.8375 RandomState 380
Test 0.8875 Train 0.8375 RandomState 381
Test 0.8875 Train 0.8375 RandomState 382
Test 0.8875 Train 0.8375 RandomState 383
Test 0.8875 Train 0.8375 RandomState 384
Test 0.8875 Train 0.8375 RandomState 385
Test 0.8875 Train 0.8375 RandomState 386
Test 0.8875 Train 0.8375 RandomState 387
Test 0.8875 Train 0.8375 RandomState 388
Test 0.8875 Train 0.8375 RandomState 389
Test 0.8875 Train 0.8375 RandomState 390
Test 0.8875 Train 0.8375 RandomState 391
Test 0.8875 Train 0.8375 RandomState 392
Test 0.8875 Train 0.8375 RandomState 393
Test 0.8875 Train 0.8375 RandomState 394
Test 0.8875 Train 0.8375 RandomState 395
Test 0.8875 Train 0.8375 RandomState 396
Test 0.8875 Train 0.8375 RandomState 397
Test 0.8875 Train 0.8375 RandomState 398
Test 0.8875 Train 0.8375 RandomState 399
Test 0.8875 Train 0.8375 RandomState 400
```

```
x_train,x_test,y_train,y_test=train_test_split(features,labels,test_size=0.2,random_state=42)
finalModel=LogisticRegression()
finalModel.fit(x_train,y_train)
```

```
LogisticRegression()
```

```
print(finalModel.score(x_test,y_test))
print(finalModel.score(x_train,y_train))
```

```
0.8875
0.8375
```

```
from sklearn.metrics import classification_report
print(classification_report(labels,finalModel.predict(features)))
```

```
              precision    recall  f1-score   support

           0       0.85      0.93      0.89       257
           1       0.85      0.70      0.77       143

    accuracy                           0.85       400
   macro avg       0.85      0.81      0.83       400
weighted avg       0.85      0.85      0.84       400
```

Result:

Thus the python program to perform the model classification using Logistic Regression is executed and verified successfully

EXPERIMENT – 10

K-MEANS CLUSTERING

Aim:

To perform model clustering using K-means Clustering technique

Procedure:

1. Upload the given dataset
2. Import all the necessities
3. Read the dataset as Data Frame
4. Using seaborn visualize the trends
5. Using sklearn train the model for predicting

Program:

```
from google.colab import files
uploaded=files.upload()
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
file=next(iter(uploaded))
df=pd.read_csv(file)
df.info
```

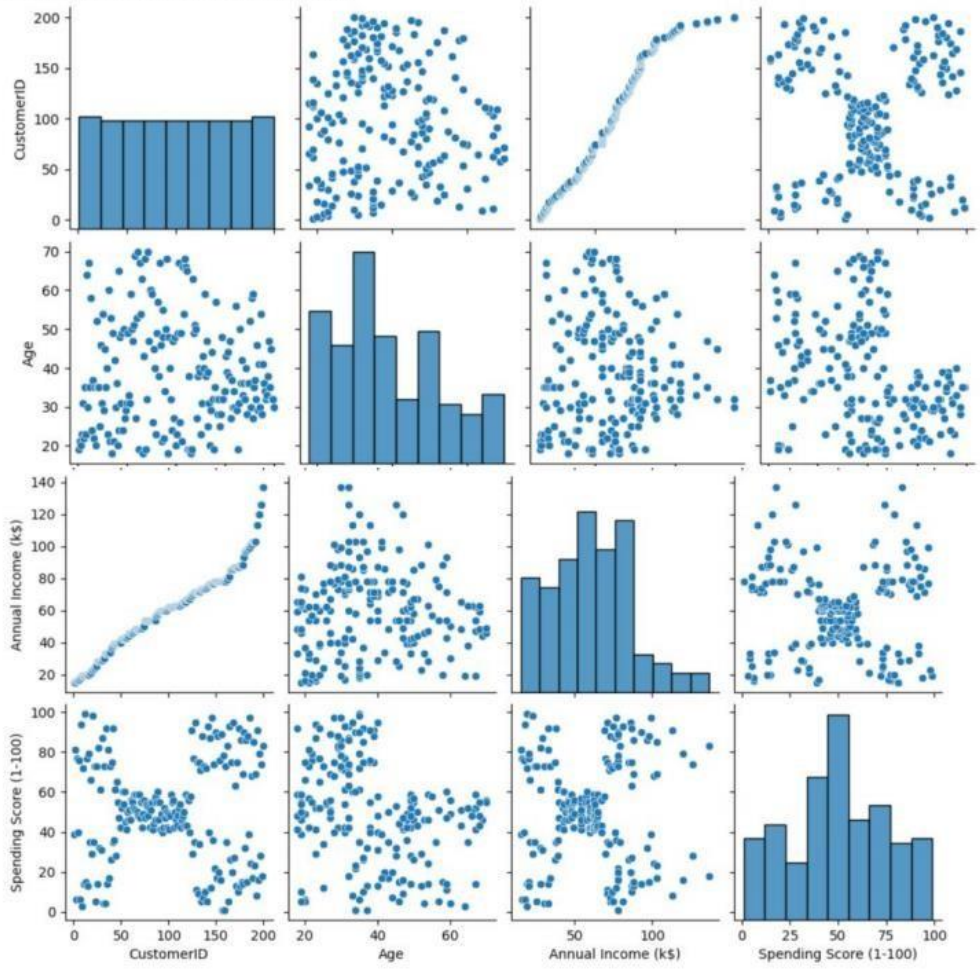
```
Choose Files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Mall_Customers - Mall_Customers.csv to Mall_Customers - Mall_Customers.csv
pandas.core.frame.DataFrame.info
def info(verbose: bool | None=None, buf: WriteBuffer[str] | None=None, max_cols: int | None=None,
memory_usage: bool | str | None=None, show_counts: bool | None=None) -> None

Print a concise summary of a DataFrame.

This method prints information about a DataFrame including
the index dtype and columns, non-null values and memory usage.

Parameters
```

```
df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
sns.pairplot(df)
```

```
<seaborn.axisgrid.PairGrid at 0x7ef83f0bb860>
```




```
features=df.iloc[:,[3,4]].values
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)
```

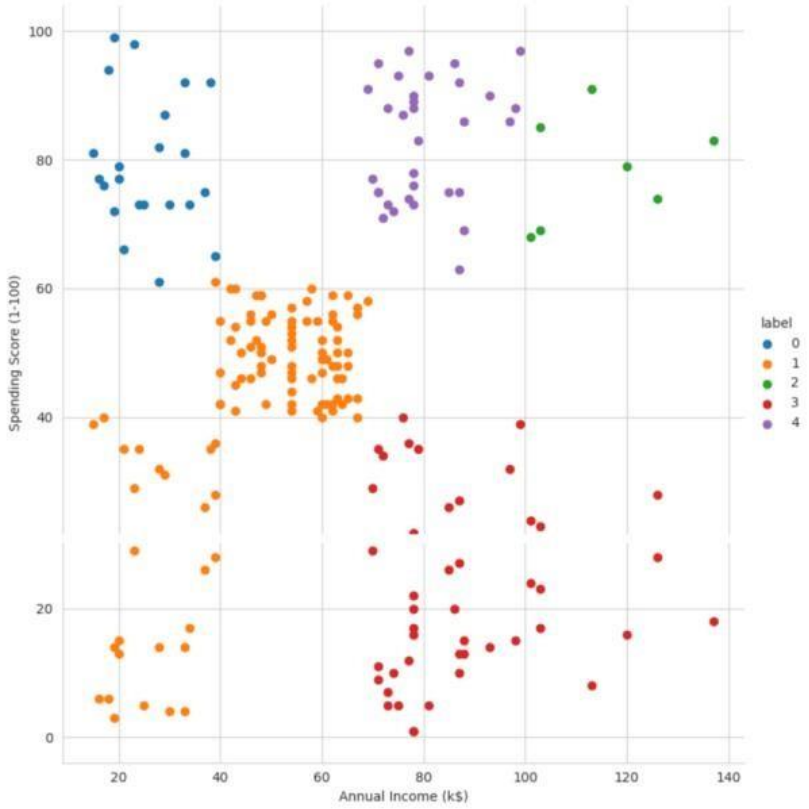
KMeans(n_clusters=5)

```
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()
```

```
/tmp/ipython-input-470183701.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  Final['label']=model.predict(features)
```

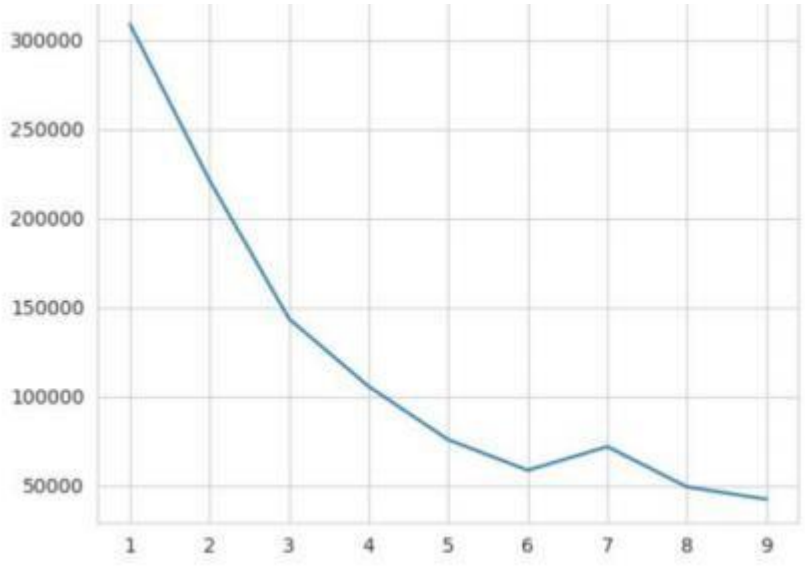
	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	1
1	15	81	0
2	16	6	1
3	16	77	0
4	17	40	1

```
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8)\
.map(plt.scatter,"Annual Income (k$)","Spending Score (1-100)")\
.add_legend();
plt.show()
```




```
features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
  model=KMeans(n_clusters=i)
  model.fit(features_el)
  wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
```

```
[<matplotlib.lines.Line2D at 0x7ef8315c1bb0>]
```



Result:

Thus the python program to make prediction model using K-means Clustering is executed and verified successfully

EXPERIMENT-11

Random Sampling and Sampling Distribution Aim:

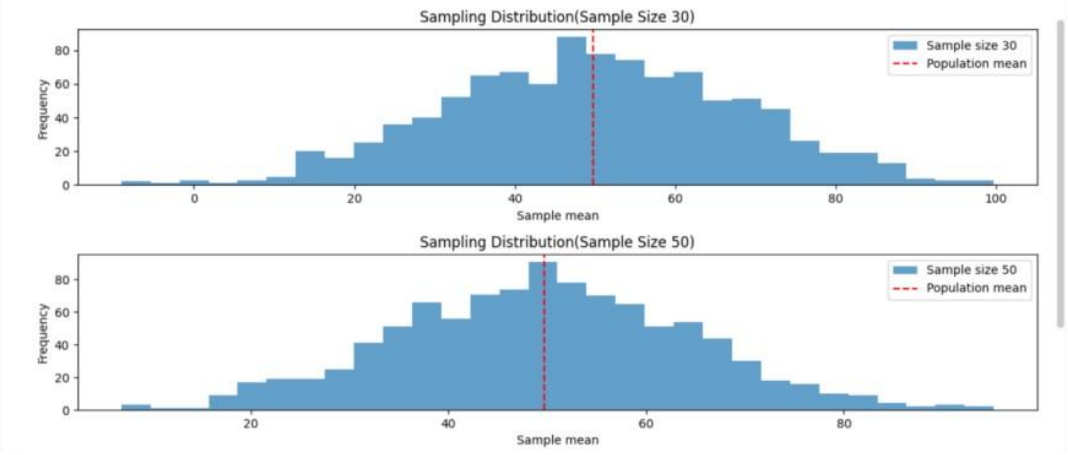
To explore random sampling from population and understand the concept of sampling distribution

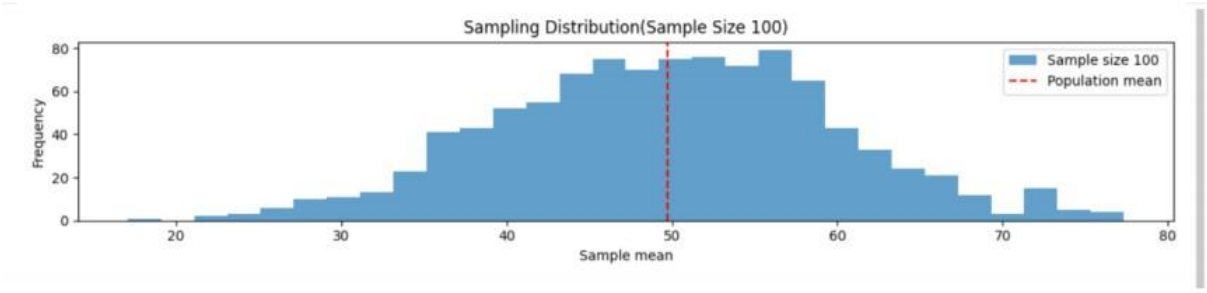
Procedure:

1. Create a population of data with specified distribution
2. Perform random sampling from population to create multiple sample of different sizes
3. Compute sample statistics
4. Plot histograms or density plots of each statistics
5. Compare the sampling distribution of the sample statistic of the sample statistic with mean distribution

Program:

```python
import numpy as np
import matplotlib.pyplot as plt
mean=50
std=100
size=100000
population=np.random.normal(mean,std,size)
sample=[30,50,100]
num=1000
sample_means={}
for i in sample:
  sample_means[i]=[]
  for j in range(num):
    samp=np.random.choice(population,size=i,replace=False)
    sample_means[i].append(np.mean(samp))
plt.figure(figsize=(12,8))
for i,j in enumerate(sample):
  plt.subplot(len(sample),1,i+1)
  plt.hist(sample_means[j],bins=30,alpha=0.7,label=f'Sample size {j}')
  plt.axvline(np.mean(population),color='red',linestyle='dashed',linewidth=1.5,label='Population mean')
  plt.title(f'Sampling Distribution(Sample Size {j})')
  plt.xlabel('Sample mean')
  plt.ylabel('Frequency')
  plt.legend()

plt.tight_layout()
plt.show()
```





Result:

Thus the python program for random sampling and sampling distribution is executed and output is verified successfully

EXPERIMENT -12

Aim:

Hypothetical using Z-Test

To test whether the average weight of species of bird differs from 150 grams

Procedure:

1. Null hypothesis
2. Alternative hypothesis
3. Sample
4. Z-Test
5. Decision Rule

Program:

```
import numpy as np
import scipy.stats as stats
sample_data = np.array([152, 148, 151, 149, 147, 153, 150, 148, 152,
149,151, 150, 149, 152, 151, 148, 150, 152, 149, 150,148, 153, 151,
150, 149, 152, 148, 151, 150, 153])
mean=150
sample_mean=np.mean(sample_data)
std=np.std(sample_data,ddof=1)
n=len(sample_data)
z_stat=(sample_mean-mean)/(std/np.sqrt(n))
p_value=2*(1-stats.norm.cdf(np.abs(z_stat)))
print(f"Sample Mean: {sample_mean:.2f}")
print(f"z-Statistic: {z_stat:.4f}")
print(f"P-value: {p_value:.4f}")
alpha=0.05
if p_value<alpha:
  print("Reject the null hypothesis:The average weight is significantly different from 150 grams")
else:
  print("Fail to reject the null hypotheis:There is no significant difference in average weight from 150 grams.")
```

```
Sample Mean: 150.20
z-Statistic: 0.6406
P-value: 0.5218
Fail to reject the null hypotheis:There is no significant difference in average weight from 150 grams.
```

Result:

Thus the python program for doing hypothetical test using Z-Test is executed and output is verified successfully

Hypothetical using T-Test

EXPERIMENT -13

Aim:

To test whether the average IQ score of a sample of students differs significantly from a

population mean IQ score of 100

Procedure:

1. Null hypothesis
2. Alternative hypothesis
3. Sample
4. Z-Test
5. Decision Rule

Program:

```
import numpy as np
import scipy.stats as stats
np.random.seed(42)
size=25
data=np.random.normal(loc=102,scale=15,size=size)
mean=100
sample_mean=np.mean(data)
std=np.std(data,ddof=1)
n=len(data)
t_stat,p_value=stats.ttest_1samp(data,mean)
print(f"Sample mean:{sample_mean:.2f}")
print(f"T-statistic:{t_stat:.4f}")
print(f"P-value:{p_value:.4f}")
alpha=0.05
if p_value < alpha :
  print("Reject the null hypothesis.The average scoreIQ score is significantly different from 100.")
else:
  print("Fail to reject the null hypothesis:There is no significant difference in average IQ score from 100")
```

```
Sample mean:99.55
T-statistic:-0.1577
P-value:0.8760
Fail to reject the null hypothesis:There is no significant difference in average IQ score from 100
```

Result:

Thus the python program for hypothetical test using T-Test is executed and output is verified successfully

Hypothetical using ANOVA Test

EXPERIMENT -14

Aim:

To compare the growth rates of plants under three different fertilizer treatments(Treatment A,B,C) to determine if there is a significant difference in their mean growth.

Procedure:

1. Null hypothesis
2. Alternative hypothesis
3. Sample
4. ANOVA
5. Decision Rule

Program:

```
import numpy as np
import scipy.stats as stats
np.random.seed(42)
n_plants=25
a=np.random.normal(loc=10,scale=2,size=n_plants)
b=np.random.normal(loc=12,scale=3,size=n_plants)
c=np.random.normal(loc=15,scale=2.5,size=n_plants)
d=np.concatenate([a,b,c])
tl=['A']*n_plants+['B']*n_plants+['C']*n_plants
fs,pv=stats.f_oneway(a,b,c)
print("Treatment A Mean Growth: ",np.mean(a))
print("Treatment B Mean Growth: ",np.mean(b))
print("Treatment C Mean Growth: ",np.mean(c))
print()
print(f"F-statistic : {fs:.4f}")
print(f"P-value : {pv:.4f}")
alpha=0.05
if pv<alpha:
  print("Reject the null hypothesis:There is a significant differnece in mean growth ratesamong three treatments")
else:
  print("Fail to reject the null hypothesis: There is no significant difference in mean growth among three treatme

if pv<alpha:
  from statsmodels.stats.multicomp import pairwise_tukeyhsd
  tuckey_results=pairwise_tukeyhsd(d,tl,alpha=0.05)
  print("\nTukey'sHSD Post-hoc test:",tuckey_results)
```

```
Treatment A Mean Growth:  9.672983882683818
Treatment B Mean Growth:  11.137680744437432
Treatment C Mean Growth:  15.265234904828972

F-statistic : 36.1214
P-value : 0.0000
Reject the null hypothesis:There is a significant differnece in mean growth ratesamong three treatments

Tukey'sHSD Post-hoc test: Multiple Comparison of Means - Tukey HSD, FWER=0.05
=====================================================
group1 group2 meandiff p-adj   lower  upper  reject
-----------------------------------------------------
     A      B   1.4647 0.0877 -0.1683 3.0977  False
     A      C   5.5923    0.0  3.9593 7.2252   True
     B      C   4.1276    0.0  2.4946 5.7605   True
-----------------------------------------------------
```

Result:

Thus the python program for hypothetical using ANOVA test is executed and output verified successfully